Тема 05. Тестирование

Блок 1 · Предыдущий: 04 · Следующий: 06. CLI-прототип

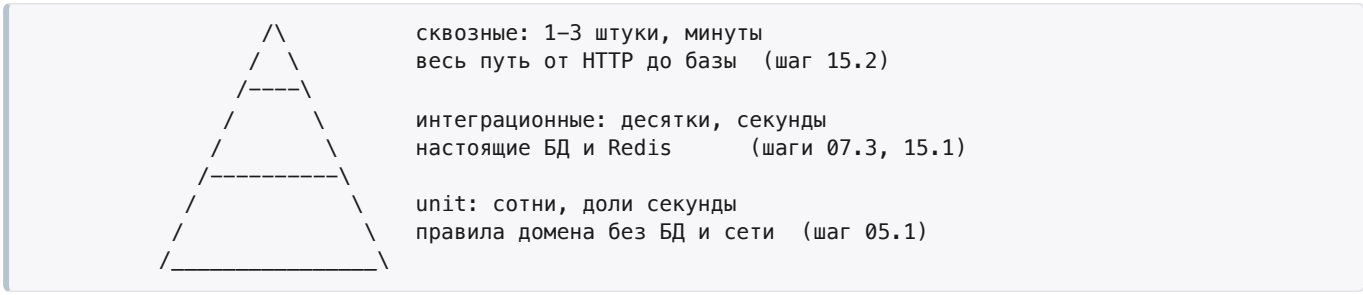
> **Время:** 10–15 часов на два шага — 05.1 (5–7 ч), 05.2 (5–8 ч).
> Обоснование — TIME.md. Незнакомые слова — словарь.

Зачем эта тема

Тесты — единственный способ менять код без страха. Дальше проект будет расти каждый шаг, и без тестов каждое изменение начнёт ломать предыдущие.

Теория

- **pytest** — стандарт тестирования в Python: тест это обычная функция с `assert`, без классов и лишних конструкций.
- **Фикстуры** — переиспользуемая подготовка данных и окружения для тестов; объявляются через декоратор и запрашиваются аргументом теста. Общие фикстуры живут в `conftest.py`.
- **Параметризация** — один тест прогоняется на наборе входных данных вместо копирования кода.
- **Тест-дубли** — подставные объекты вместо настоящих зависимостей: mock записывает вызовы, stub возвращает заготовленный ответ, fake содержит упрощённую рабочую реализацию, spy наблюдает за настоящим объектом. Fake обычно лучше остальных: он проверяет поведение, а не последовательность вызовов.
- **Пирамида тестов** — много быстрых unit-тестов, меньше интеграционных, совсем мало сквозных.



Чем выше уровень, тем медленнее и дороже тест и тем труднее по его падению понять, что именно сломалось. Поэтому основную проверку правил делают внизу, а вверху оставляют несколько сценариев, подтверждающих, что всё собрано вместе правильно.

- **Покрытие** — доля строк, выполненных во время тестов. Индикатор непроверенных мест, а не цель. 100% покрытия не означает отсутствия ошибок.

Как pytest находит и запускает тесты

pytest собирает файлы `test_*.py`, в них — функции `test_*`. При падении `assert` pytest печатает значения обеих частей сравнения. Ключи запуска: `-x` — стоп на первом падении, `-k текст` — тесты с текстом в имени, `--lf` — только упавшие в прошлый раз.

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra --strict-markers"   # неизвестный маркер — ошибка, а не опечатка
markers = ["integration: тесты с настоящей БД и отдельными процессами"]
```

Структура теста: подготовка, действие, проверка

```
def test_rejected_application_is_final(application):
    application.move_to(Status.REJECTED)                # подготовка

    with pytest.raises(TransitionError, match="rejected"):
        application.move_to(Status.APPLIED)             # действие

    assert application.status is Status.REJECTED        # проверка
```

Фикстуры

```
@pytest.fixture
def repo() -> InMemoryRepository:
    return InMemoryRepository()          # новый объект на каждый тест

@pytest.fixture
def data_file(tmp_path):                 # tmp_path — встроенная фикстура
    path = tmp_path / "records.json"
    path.write_text("[]", encoding="utf-8")
    yield path                           # здесь выполняется тест
    # код после yield — teardown, выполнится и при падении теста
```

Тест запрашивает фикстуру именем аргумента, фикстуры могут запрашивать друг друга. Область видимости (`scope`) задаёт, как часто фикстура создаётся: `function` по умолчанию — на каждый тест, `module` — на файл, `session` — на весь прогон. Общие фикстуры кладут в `conftest.py`: pytest находит его сам, импортировать не нужно. Из встроенных чаще других нужны `tmp_path`, `monkeypatch` (временно задать переменную окружения или атрибут) и `caplog` (перехватить записи логов).

Параметризация

```
@pytest.mark.parametrize(
    ("current", "new"),
    [(Status.SAVED, Status.APPLIED), (Status.APPLIED, Status.INTERVIEW)],
    ids=["saved-applied", "applied-interview"],
)
def test_allowed_transition(current, new):
    application = Application(vacancy_id=1, status=current)
    application.move_to(new)
    assert application.status is new
```

Каждый набор значений — отдельный тест со своим именем в отчёте. Если для разных наборов внутри теста появляется `if`, это уже разные сценарии: их пишут отдельными тестами.

Fake и mock на одном примере

```
class FakeNotifier:                        # fake: упрощённая рабочая реализация
    def __init__(self) -> None:
        self.messages: list[str] = []

    def notify(self, message: str) -> None:
        self.messages.append(message)

def test_status_change_notifies():
    notifier = FakeNotifier()
    change_status(application, Status.APPLIED, notifier)
    assert notifier.messages == ["Отклик 1: saved -> applied"]

def test_status_change_notifies_with_mock():
    notifier = Mock()                      # mock: записывает вызовы
    change_status(application, Status.APPLIED, notifier)
    notifier.notify.assert_called_once_with("Отклик 1: saved -> applied")
```

Первый тест проверяет результат, второй — факт вызова с аргументами. Mock уместен, когда взаимодействие и есть требование: «отправили ровно одно уведомление». Если мокать всё подряд, тест начинает повторять реализацию и ломается при любом рефакторинге. `unittest.mock.patch` подменяет имя там, где его **используют**: для функции `send`, импортированной в модуль `service`, патчат `"job_tracker.service.send"`, а не модуль, где она объявлена. Мокать не нужно чистые функции, доменные объекты и всё, что быстро и детерминированно работает само.

Покрытие и проверка самих тестов

`pytest --cov=job_tracker --cov-report=term-missing` печатает процент по файлам и номера невыполненных строк. Строка считается покрытой, даже если результат её работы ничем не проверен: тест без единого `assert` тоже даёт покрытие. `--cov-branch` дополнительно проверяет, что каждое условие проходилось в обе стороны. Намеренная поломка условия в бизнес-логике — ручная версия мутационного тестирования: если ни один тест не упал, это место фактически не проверено, сколько бы процентов ни показывал отчёт.

При исправлении бага сначала пишут тест, воспроизводящий ошибку, и видят, что он падает именно по этой причине. Затем исправляют код и видят зелёный тест. Он остаётся в наборе и не даёт ошибке вернуться — поэтому его называют регрессионным.

Git: stash, cherry-pick, reflog, bisect

- `git stash` откладывает незакоммиченные изменения и очищает рабочую папку; `git stash pop` возвращает их. Нужно, чтобы быстро переключиться на другую ветку.
- `git cherry-pick <хеш>` копирует один коммит на текущую ветку — например, исправление из чужой ветки без её остальных изменений.
- `git reflog` — журнал всех положений `HEAD`, включая «потерянные» после `reset` или `rebase`. Коммит из него возвращают командой `git reset --hard HEAD@{2}` или веткой на хеш.
- `git bisect` ищет коммит, в котором появилась регрессия, делением пополам:

```
хороший                                                  плохой
  c1 ── c2 ── c3 ── c4 ── c5 ── c6 ── c7 ── c8 ── c9
                           ^ проверка 1: плохо  -> ищем левее
                ^ проверка 2: хорошо -> ищем правее
                       ^ проверка 3: плохо  -> виновник c4
```

Вы отмечаете известный хороший и плохой коммиты (`git bisect start`, `git bisect bad`, `git bisect good c1`), Git переключает на середину, вы проверяете и отвечаете `good` или `bad`. Команда `git bisect run uv run pytest tests/test_status.py` делает проверки сама по коду возврата теста. Каждая проверка вдвое сужает диапазон: среди тысячи коммитов виновник находится примерно за десять шагов. В конце — `git bisect reset`.

Материалы

- pytest: How-to guides — фикстуры, параметризация, проверка исключений.
- pytest: Fixtures — области видимости и порядок выполнения.
- unittest.mock — `Mock`, `patch`, `assert_called_with`.
- pytest-cov — запуск с измерением покрытия.

Задание

Шаг 05.1. Первые тесты домена (5–7 ч)

1. Настройте pytest в `pyproject.toml`: пути к тестам, опции по умолчанию, строгие маркеры.
2. Напишите unit-тесты на доменную логику из шагов 03 и 04: преобразования, валидацию, правила.
3. Используйте параметризацию для проверки набора случаев одним тестом.
4. Вынесите подготовку данных в фикстуры в `conftest.py`.
5. Протестируйте выбор реализации через фабрику, подставляя заглушку из шага 04.1.
6. Обязательно напишите негативные тесты: некорректные данные, отсутствующие значения, ошибки источника.
7. Добавьте тесты переходов состояния: разрешённый, запрещённый, неизменность после ошибки и повторный переход в то же состояние.

Шаг 05.2. Покрытие, поиск дефекта и Git (5–8 ч)

1. Подключите измерение покрытия и доведите покрытие доменного кода минимум до 60%. Довести до 70% — желательный критерий, а не условие закрытия шага.
2. Проверьте качество тестов: намеренно сломайте условие в бизнес-логике и убедитесь, что тест упал. Верните условие обратно. Это дешёвая проверка того, что тесты действительно что-то проверяют.
3. Зафиксируйте баг в issue, воспроизведите его, сначала получите падающий тест, затем исправьте код.
4. Добавьте `uv run pytest -m "not integration"` в существующий CI и зарегистрируйте маркер `integration` заранее. Тесты базы данных получат этот маркер на шаге 07.3.
5. Выполните упражнение с Git и регрессией из WORKFLOW: `git stash`, `git cherry-pick`, `git reflog` и поиск намеренно добавленной регрессии через `git bisect`.
6. **По желанию:** объясните отличия mock, stub, fake и spy своими словами и примените `unittest.mock` там, где нужно проверить именно взаимодействие, а не результат.

Обязательные критерии

- ☐ `pytest` проходит локально, прогон занимает считанные секунды.

- ▢ Unit-тесты домена не обращаются в сеть, к файловой системе и БД. Тест адаптера файла, если он есть, использует временную директорию и отделён от unit-тестов домена.
- ▢ Есть и позитивные, и негативные сценарии.
- ▢ Хотя бы один тест параметризован, общие данные вынесены в фикстуры.
- ▢ Покрытие доменного кода не ниже 60%, отчёт о покрытии воспроизводим командой.
- ▢ Тесты не зависят от порядка выполнения и проходят при запуске поодиночке.
- ▢ Падающий и исправленный сценарии бага зафиксированы в issue/PR; тесты выполняются в CI.
- ▢ Проверено, что тест падает при намеренной поломке бизнес-логики.

Желательные критерии

- ▢ Покрытие доменного кода доведено до 70% и выше.
- ▢ Выполнено упражнение с `git stash`, `cherry-pick`, `reflog` и `bisect`.
- ▢ Объяснены отличия mock, stub, fake и spy; `unittest.mock` применён по назначению.

На что обратить внимание

- Тест проверяет поведение, а не реализацию: после рефакторинга без смены поведения он должен остаться зелёным.
- Один проверяемый сценарий на тест; связанных `assert` может быть несколько.
- Имя теста описывает проверяемое условие: `test_reject_expired_item`, а не `test_1`.
- Гнаться за процентом покрытия бессмысленно. Лучше пять тестов на важные правила, чем двадцать на геттеры.

Вопросы для собеседования

Проверьте себя по темам этого шага. Ответов в программе нет намеренно: формулировка своими словами и есть проверка. Где можно — отвечайте на примере своего кода.

1. Что такое фикстура и чем она лучше подготовки данных внутри теста?
2. Зачем нужна параметризация и когда она хуже отдельных тестов?
3. Чем отличаются mock, stub, fake и spy? Что вы предпочитаете и почему?
4. Когда мокать зависимость не нужно?
5. Что такое пирамида тестов? Почему сквозных тестов делают мало?
6. Что показывает покрытие кода и что оно не показывает?
7. Как проверить, что тесты действительно что-то проверяют?
8. Зачем писать негативные тесты, если позитивные проходят?
9. Почему тест не должен зависеть от порядка запуска? Как проявляется утечка состояния?
10. Зачем сначала получать падающий тест, а потом исправлять код?
11. Что такое маркеры pytest и зачем разделять быстрые и медленные тесты?
12. Тест проверяет поведение или реализацию? Что должно случиться с тестом после рефакторинга?
13. Как найти коммит, в котором появилась регрессия, если известны рабочая и сломанная версии?
14. Что делает `git stash` и в какой ситуации спасает `git reflog`?

Полный набор по блоку — вопросы по блоку 1.